main fruit game_engine

consider the attached program files

The codebase represents a Python Pygame clone of **Fruit Ninja**. PY + 2

Here is a quick overview of the structure and main components:

Key Components

`main.py` : Initializes Pygame, sets up the window resolution (700×600), creates the `GameEngine` instance, and runs the standard 60 FPS event/render loop. PY

`fruit.py` : Defines the `Fruit` class representing both fruits and bombs. Tracks position, velocity (v_x, v_y), gravity, radius, type, and slicing state. Includes collision detection via `contains_point(x, y)` and off-screen detection. PY + 1

`game_engine.py` : Manages the core game state: PY

- Spawns fruits/bombs periodically. PY
- Handles mouse motion events to detect slices and record mouse positions for rendering the blade trail. PY
- Updates fruit physics, removes off-screen or sliced objects, and updates player lives and score. PY
- Draws fruits, blade trail, and HUD score/lives onto the screen. PY

Known Issues / Todo Notes in the Code

1. **Slice Collision Glitch (`fruit.py` / `game_engine.py`):**
   `contains_point(x, y)` only checks single discrete points against a fruit's circle. Fast swipe motions skip intervening space across frames, allowing the mouse to pass

completely through fruits without triggering a hit. Line segment-to-circle intersection checking between consecutive trail points is required to fix this. PY + 3

2. **Missing Game Over Screen (`game_engine.py`):**
When `game_over` becomes `True` , the engine simply stops updating objects and prints the score to the console once. There is currently no visual "Game Over"

help modify the fruit and game engine files to fix the collision glitch caused by fast movements

To fix the fast-swipe collision glitch, we need to check line segment intersection between consecutive mouse positions rather than checking single point hits. PY

Implementation Plan

1. `fruit.py` : Add a `intersects_segment(p1, p2)` method to check if the line segment between two consecutive mouse points `p1` and `p2` intersects the fruit's circle. PY
2. `game_engine.py` : Update `_handle_motion` to check segment collision using the latest mouse segment (`self.trail[-1]` to `pos`) instead of just testing single points . PY + 1

1. Updated `fruit.py`

Python

```python
import math

class Fruit:
    def __init__(self, x, y, vx, vy, gravity, radius=28, kind="fruit"):
        self.x = x
        self.y = y
        self.vx = vx
        self.vy = vy
```

```
        self.gravity = gravity
        self.radius = radius
        self.kind = kind  # "fruit" or "bomb"
        self.sliced = False

    def update(self):
        self.vy += self.gravity
        self.x += self.vx
        self.y += self.vy

    def contains_point(self, x, y):
        return math.hypot(self.x - x, self.y - y) <= self.radius

    def intersects_segment(self, p1, p2):
        """Checks if the line segment from p1 to p2 intersects this fruit's circl
        x1, y1 = p1
        x2, y2 = p2

        # Vector from p1 to p2
        dx = x2 - x1
        dy = y2 - y1

        # Length squared of segment
        len_sq = dx * dx + dy * dy
        if len_sq == 0:
            return self.contains_point(x1, y1)

        # Projection parameter t of circle center onto line segment [0, 1]
        t = max(0.0, min(1.0, ((self.x - x1) * dx + (self.y - y1) * dy) / len_sq)

        # Closest point on the segment to the fruit's center
        closest_x = x1 + t * dx
        closest_y = y1 + t * dy

        # Distance from closest point to fruit center
        return math.hypot(self.x - closest_x, self.y - closest_y) <= self.radius

    def off_screen(self, height):
        return self.y - self.radius > height
```

2. Updated `game_engine.py`

Python

```python
import pygame
import random
from .fruit import Fruit

WHITE = (255, 255, 255)
BOMB_BLACK = (30, 30, 30)
FRUIT_COLORS = [(220, 60, 60), (230, 140, 40), (230, 200, 40), (90, 180, 90)]

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.fruits = []
        self.trail = []  # recent mouse positions, drawn as the "blade"

        self.spawn_interval = 55  # frames between spawns
        self._spawn_timer = 0
        self.bomb_chance = 0.15
        self.speed_scale = 1.0

        self.lives = 3
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.game_over = False

    def spawn_fruit(self):
        x = random.randint(60, self.width - 60)
        vy = -random.uniform(13, 16) * self.speed_scale
        vx = random.uniform(-2, 2)
        gravity = 0.35
        kind = "bomb" if random.random() < self.bomb_chance else "fruit"

        fruit = Fruit(x, self.height + 30, vx, vy, gravity, kind=kind)
        fruit.color = BOMB_BLACK if kind == "bomb" else random.choice(FRUIT_COLOR
        self.fruits.append(fruit)

    def handle_event(self, event):
        if event.type == pygame.MOUSEMOTION:
            self._handle_motion(event.pos)

    def _handle_motion(self, pos):
        # If we have a previous trail point, test segment intersection
```

```
        if self.trail:
            last_pos = self.trail[-1]
            for fruit in self.fruits:
                if not fruit.sliced and fruit.intersects_segment(last_pos, pos):
                    self._slice(fruit)
        else:
            # Fallback point check for the very first mouse position
            for fruit in self.fruits:
                if not fruit.sliced and fruit.contains_point(pos[0], pos[1]):
                    self._slice(fruit)

        self.trail.append(pos)
        if len(self.trail) > 15:
            self.trail.pop(0)

    def _slice(self, fruit):
        fruit.sliced = True
        if fruit.kind == "bomb":
            self.game_over = True
        else:
            self.score += 1

    def handle_input(self):
        pass

    def update(self):
        if self.game_over:
            return

        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.spawn_fruit()

        still_alive = []
        for fruit in self.fruits:
            fruit.update()
            if fruit.sliced:
                continue
            if fruit.off_screen(self.height):
                if fruit.kind == "fruit":
                    self.lives -= 1
                continue
            still_alive.append(fruit)
        self.fruits = still_alive
```

```
        if self.lives <= 0:
            self.game_over = True

    def render(self, screen):
        for fruit in self.fruits:
            color = getattr(fruit, "color", WHITE)
            pygame.draw.circle(screen, color, (int(fruit.x), int(fruit.y)), fruit

        if len(self.trail) >= 2:
            pygame.draw.lines(screen, WHITE, False, self.trail, 3)

        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))
        lives_text = self.font.render(f"Lives: {self.lives}", True, WHITE)
        screen.blit(lives_text, (self.width - 130, 10))

        if self.game_over and not getattr(self, "_game_over_logged", False):
            print("Game over! Final score:", self.score)
            self._game_over_logged = True
```

instead of printing final score into console, add a game over screen that displays the final score when the player runs out of lives or if the player hits a bomb

Here are the updated `game_engine.py` and `main.py` files to add a visual Game Over screen with a restart prompt when the player runs out of lives or hits a bomb. PY

Summary of Changes

1. `game_engine.py` :

   Added a `restart()` method to reset score, lives, trail, fruits, and timers. PY

   Rendered a centered "GAME OVER" banner, the final score, and a "Press SPACE or CLICK to Restart" prompt when `game_over` is `True` . PY

   Updated `handle_event` to listen for Space key or mouse click during game over to trigger a reset. PY + 1

2. `main.py` :

   Ensured game engine updates continue running so screen refreshes and state resets properly. PY

1. Updated `game_engine.py`

Python

```python
import pygame
import random
from .fruit import Fruit

WHITE = (255, 255, 255)
RED = (230, 60, 60)
GRAY = (180, 180, 180)
BOMB_BLACK = (30, 30, 30)
FRUIT_COLORS = [(220, 60, 60), (230, 140, 40), (230, 200, 40), (90, 180, 90)]

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.fruits = []
        self.trail = []

        self.spawn_interval = 55
        self._spawn_timer = 0
        self.bomb_chance = 0.15
        self.speed_scale = 1.0

        self.lives = 3
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.title_font = pygame.font.SysFont("Arial", 54, bold=True)
        self.subtitle_font = pygame.font.SysFont("Arial", 22)
        self.game_over = False

    def restart(self):
        self.fruits.clear()
        self.trail.clear()
        self.lives = 3
        self.score = 0
```

```
        self._spawn_timer = 0
        self.game_over = False

    def spawn_fruit(self):
        x = random.randint(60, self.width - 60)
        vy = -random.uniform(13, 16) * self.speed_scale
        vx = random.uniform(-2, 2)
        gravity = 0.35
        kind = "bomb" if random.random() < self.bomb_chance else "fruit"

        fruit = Fruit(x, self.height + 30, vx, vy, gravity, kind=kind)
        fruit.color = BOMB_BLACK if kind == "bomb" else random.choice(FRUIT_COLOR
        self.fruits.append(fruit)

    def handle_event(self, event):
        if self.game_over:
            if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
                self.restart()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                self.restart()
            return

        if event.type == pygame.MOUSEMOTION:
            self._handle_motion(event.pos)

    def _handle_motion(self, pos):
        if self.trail:
            last_pos = self.trail[-1]
            for fruit in self.fruits:
                if not fruit.sliced and fruit.intersects_segment(last_pos, pos):
                    self._slice(fruit)
        else:
            for fruit in self.fruits:
                if not fruit.sliced and fruit.contains_point(pos[0], pos[1]):
                    self._slice(fruit)

        self.trail.append(pos)
        if len(self.trail) > 15:
            self.trail.pop(0)

    def _slice(self, fruit):
        fruit.sliced = True
        if fruit.kind == "bomb":
            self.game_over = True
        else:
            self.score += 1
```

```
def handle_input(self):
    pass

def update(self):
    if self.game_over:
        return

    self._spawn_timer += 1
    if self._spawn_timer >= self.spawn_interval:
        self._spawn_timer = 0
        self.spawn_fruit()

    still_alive = []
    for fruit in self.fruits:
        fruit.update()
        if fruit.sliced:
            continue
        if fruit.off_screen(self.height):
            if fruit.kind == "fruit":
                self.lives -= 1
            continue
        still_alive.append(fruit)
    self.fruits = still_alive

    if self.lives <= 0:
        self.game_over = True

def render(self, screen):
    # Render active fruits and blade trail
    for fruit in self.fruits:
        color = getattr(fruit, "color", WHITE)
        pygame.draw.circle(screen, color, (int(fruit.x), int(fruit.y)), fruit

    if len(self.trail) >= 2:
        pygame.draw.lines(screen, WHITE, False, self.trail, 3)

    # HUD
    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))
    lives_text = self.font.render(f"Lives: {self.lives}", True, WHITE)
    screen.blit(lives_text, (self.width - 130, 10))

    # Game Over Overlay
    if self.game_over:
        # Semi-transparent dark overlay
```

```
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))

        # Game Over text
        go_surf = self.title_font.render("GAME OVER", True, RED)
        go_rect = go_surf.get_rect(center=(self.width // 2, self.height // 2
        screen.blit(go_surf, go_rect)

        # Final Score text
        score_surf = self.font.render(f"Final Score: {self.score}", True, WHI
        score_rect = score_surf.get_rect(center=(self.width // 2, self.height
        screen.blit(score_surf, score_rect)

        # Restart instruction
        restart_surf = self.subtitle_font.render("Press SPACE or CLICK to pla
        restart_rect = restart_surf.get_rect(center=(self.width // 2, self.he
        screen.blit(restart_surf, restart_rect)
```

2. Updated `main.py`

Python

```python
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 700, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Fruit Slice - Pygame Version")

# Colors
DARK_BLUE = (20, 25, 45)

# Clock
clock = pygame.time.Clock()
FPS = 60
```

```
# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(DARK_BLUE)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

after the game over, also allow the player to play again in a difficulty: easy, medium, hard (determining bomb spawn rate) or to exit

Here are the updated `game_engine.py` and `main.py` files to add an interactive Game Over screen with difficulty selection (**Easy**, **Medium**, **Hard**) and an **Exit** option. PY + 1

Difficulty Settings

Difficulty	Bomb Spawn Chance
Easy	5%

Difficulty	Bomb Spawn Chance
Medium	15% *(Default)*
Hard	30%

1. Updated `game_engine.py`

Python

```python
import pygame
import random
from .fruit import Fruit

WHITE = (255, 255, 255)
RED = (230, 60, 60)
GRAY = (180, 180, 180)
YELLOW = (240, 210, 60)
GREEN = (90, 200, 90)
ORANGE = (230, 140, 40)
BOMB_BLACK = (30, 30, 30)
FRUIT_COLORS = [(220, 60, 60), (230, 140, 40), (230, 200, 40), (90, 180, 90)]

# Difficulty settings mapping to bomb spawn chance
DIFFICULTIES = {
    "EASY": {"bomb_chance": 0.05, "color": GREEN},
    "MEDIUM": {"bomb_chance": 0.15, "color": ORANGE},
    "HARD": {"bomb_chance": 0.30, "color": RED},
}

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.fruits = []
        self.trail = []

        self.spawn_interval = 55
        self._spawn_timer = 0
        self.difficulty = "MEDIUM"
```

```
        self.bomb_chance = DIFFICULTIES[self.difficulty]["bomb_chance"]
        self.speed_scale = 1.0

        self.lives = 3
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.title_font = pygame.font.SysFont("Arial", 50, bold=True)
        self.button_font = pygame.font.SysFont("Arial", 22, bold=True)

        self.game_over = False
        self.should_quit = False  # Set to True when player clicks Exit
        self.button_rects = {}

    def set_difficulty(self, mode):
        if mode in DIFFICULTIES:
            self.difficulty = mode
            self.bomb_chance = DIFFICULTIES[mode]["bomb_chance"]

    def restart(self, difficulty=None):
        if difficulty:
            self.set_difficulty(difficulty)
        self.fruits.clear()
        self.trail.clear()
        self.lives = 3
        self.score = 0
        self._spawn_timer = 0
        self.game_over = False

    def spawn_fruit(self):
        x = random.randint(60, self.width - 60)
        vy = -random.uniform(13, 16) * self.speed_scale
        vx = random.uniform(-2, 2)
        gravity = 0.35
        kind = "bomb" if random.random() < self.bomb_chance else "fruit"

        fruit = Fruit(x, self.height + 30, vx, vy, gravity, kind=kind)
        fruit.color = BOMB_BLACK if kind == "bomb" else random.choice(FRUIT_COLOR
        self.fruits.append(fruit)

    def handle_event(self, event):
        if self.game_over:
            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                pos = event.pos
                # Check button clicks
                for key, rect in self.button_rects.items():
                    if rect.collidepoint(pos):
```

```
                    if key in DIFFICULTIES:
                        self.restart(difficulty=key)
                    elif key == "EXIT":
                        self.should_quit = True
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_1:
                    self.restart("EASY")
                elif event.key == pygame.K_2:
                    self.restart("MEDIUM")
                elif event.key == pygame.K_3:
                    self.restart("HARD")
                elif event.key in (pygame.K_ESCAPE, pygame.K_q):
                    self.should_quit = True
            return

        if event.type == pygame.MOUSEMOTION:
            self._handle_motion(event.pos)

    def _handle_motion(self, pos):
        if self.trail:
            last_pos = self.trail[-1]
            for fruit in self.fruits:
                if not fruit.sliced and fruit.intersects_segment(last_pos, pos):
                    self._slice(fruit)
        else:
            for fruit in self.fruits:
                if not fruit.sliced and fruit.contains_point(pos[0], pos[1]):
                    self._slice(fruit)

        self.trail.append(pos)
        if len(self.trail) > 15:
            self.trail.pop(0)

    def _slice(self, fruit):
        fruit.sliced = True
        if fruit.kind == "bomb":
            self.game_over = True
        else:
            self.score += 1

    def handle_input(self):
        pass

    def update(self):
        if self.game_over:
            return
```

```
        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.spawn_fruit()

        still_alive = []
        for fruit in self.fruits:
            fruit.update()
            if fruit.sliced:
                continue
            if fruit.off_screen(self.height):
                if fruit.kind == "fruit":
                    self.lives -= 1
                continue
            still_alive.append(fruit)
        self.fruits = still_alive

        if self.lives <= 0:
            self.game_over = True

    def render(self, screen):
        # Active fruits and blade trail
        for fruit in self.fruits:
            color = getattr(fruit, "color", WHITE)
            pygame.draw.circle(screen, color, (int(fruit.x), int(fruit.y)), fruit

        if len(self.trail) >= 2:
            pygame.draw.lines(screen, WHITE, False, self.trail, 3)

        # HUD
        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))
        lives_text = self.font.render(f"Lives: {self.lives}", True, WHITE)
        screen.blit(lives_text, (self.width - 130, 10))

        # Game Over Screen
        if self.game_over:
            # Semi-transparent overlay
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            # Header & Final Score
            go_surf = self.title_font.render("GAME OVER", True, RED)
            go_rect = go_surf.get_rect(center=(self.width // 2, 130))
```

```
screen.blit(go_surf, go_rect)

score_surf = self.font.render(f"Final Score: {self.score}", True, WHI
score_rect = score_surf.get_rect(center=(self.width // 2, 190))
screen.blit(score_surf, score_rect)

prompt_surf = self.font.render("Play Again - Select Difficulty:", Tru
prompt_rect = prompt_surf.get_rect(center=(self.width // 2, 250))
screen.blit(prompt_surf, prompt_rect)

# Draw Difficulty Buttons
self.button_rects = {}
btn_width, btn_height = 160, 48
btn_x = self.width // 2 - btn_width // 2

options = [
    ("EASY", "1. Easy (5% Bombs)", 300),
    ("MEDIUM", "2. Medium (15% Bombs)", 360),
    ("HARD", "3. Hard (30% Bombs)", 420),
]

mouse_pos = pygame.mouse.get_pos()

for key, label, y_pos in options:
    rect = pygame.Rect(btn_x, y_pos, btn_width, btn_height)
    self.button_rects[key] = rect

    # Highlight if current difficulty or hovered
    is_selected = (self.difficulty == key)
    is_hovered = rect.collidepoint(mouse_pos)
    bg_color = (60, 70, 90) if not (is_selected or is_hovered) else (
    border_color = DIFFICULTIES[key]["color"] if is_selected else WHI

    pygame.draw.rect(screen, bg_color, rect, border_radius=8)
    pygame.draw.rect(screen, border_color, rect, width=2 if not is_se

    txt_surf = self.button_button_font.render(label, True, WHITE) if
    txt_rect = txt_surf.get_rect(center=rect.center)
    screen.blit(txt_surf, txt_rect)

# Draw Exit Button
exit_rect = pygame.Rect(btn_x, 490, btn_width, btn_height)
self.button_rects["EXIT"] = exit_rect
exit_hover = exit_rect.collidepoint(mouse_pos)
exit_bg = (120, 40, 40) if exit_hover else (70, 30, 30)
```

```
            pygame.draw.rect(screen, exit_bg, exit_rect, border_radius=8)
            pygame.draw.rect(screen, RED, exit_rect, width=2, border_radius=8)

            exit_surf = self.button_font.render("Exit Game (Esc)", True, WHITE)
            exit_rect_txt = exit_surf.get_rect(center=exit_rect.center)
            screen.blit(exit_surf, exit_rect_txt)
```

2. Updated `main.py`

Python

```python
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 700, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Fruit Slice - Pygame Version")

# Colors
DARK_BLUE = (20, 25, 45)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(DARK_BLUE)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)
```

```
        # Check if user clicked Exit on the Game Over screen
        if engine.should_quit:
            running = False

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

make the text fit in the buttons/ make the buttons fit the text

Here are the modified `game_engine.py` and `main.py` files with wider, dynamically sized buttons that cleanly fit all option text and hotkey labels without clipping. PY

Key Adjustments

1. **Dynamic Button Widths**: Increased button width from 160px to 260px to comfortably fit longer text strings like `"1. Easy (5% Bombs)"`. PY
2. **Fixed Attribute Typo**: Standardized on `self.button_font` for all button label rendering. PY

1. Updated `game_engine.py`

Python

```
import pygame
import random
from .fruit import Fruit

WHITE = (255, 255, 255)
RED = (230, 60, 60)
GRAY = (180, 180, 180)
YELLOW = (240, 210, 60)
```

```
GREEN = (90, 200, 90)
ORANGE = (230, 140, 40)
BOMB_BLACK = (30, 30, 30)
FRUIT_COLORS = [(220, 60, 60), (230, 140, 40), (230, 200, 40), (90, 180, 90)]

# Difficulty settings mapping to bomb spawn chance
DIFFICULTIES = {
    "EASY": {"bomb_chance": 0.05, "color": GREEN},
    "MEDIUM": {"bomb_chance": 0.15, "color": ORANGE},
    "HARD": {"bomb_chance": 0.30, "color": RED},
}

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.fruits = []
        self.trail = []

        self.spawn_interval = 55
        self._spawn_timer = 0
        self.difficulty = "MEDIUM"
        self.bomb_chance = DIFFICULTIES[self.difficulty]["bomb_chance"]
        self.speed_scale = 1.0

        self.lives = 3
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.title_font = pygame.font.SysFont("Arial", 50, bold=True)
        self.button_font = pygame.font.SysFont("Arial", 20, bold=True)

        self.game_over = False
        self.should_quit = False
        self.button_rects = {}

    def set_difficulty(self, mode):
        if mode in DIFFICULTIES:
            self.difficulty = mode
            self.bomb_chance = DIFFICULTIES[mode]["bomb_chance"]

    def restart(self, difficulty=None):
        if difficulty:
            self.set_difficulty(difficulty)
        self.fruits.clear()
        self.trail.clear()
```

```
        self.lives = 3
        self.score = 0
        self._spawn_timer = 0
        self.game_over = False

    def spawn_fruit(self):
        x = random.randint(60, self.width - 60)
        vy = -random.uniform(13, 16) * self.speed_scale
        vx = random.uniform(-2, 2)
        gravity = 0.35
        kind = "bomb" if random.random() < self.bomb_chance else "fruit"

        fruit = Fruit(x, self.height + 30, vx, vy, gravity, kind=kind)
        fruit.color = BOMB_BLACK if kind == "bomb" else random.choice(FRUIT_COLOR
        self.fruits.append(fruit)

    def handle_event(self, event):
        if self.game_over:
            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                pos = event.pos
                for key, rect in self.button_rects.items():
                    if rect.collidepoint(pos):
                        if key in DIFFICULTIES:
                            self.restart(difficulty=key)
                        elif key == "EXIT":
                            self.should_quit = True
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_1:
                    self.restart("EASY")
                elif event.key == pygame.K_2:
                    self.restart("MEDIUM")
                elif event.key == pygame.K_3:
                    self.restart("HARD")
                elif event.key in (pygame.K_ESCAPE, pygame.K_q):
                    self.should_quit = True
            return

        if event.type == pygame.MOUSEMOTION:
            self._handle_motion(event.pos)

    def _handle_motion(self, pos):
        if self.trail:
            last_pos = self.trail[-1]
            for fruit in self.fruits:
                if not fruit.sliced and fruit.intersects_segment(last_pos, pos):
                    self._slice(fruit)
```

```
        else:
            for fruit in self.fruits:
                if not fruit.sliced and fruit.contains_point(pos[0], pos[1]):
                    self._slice(fruit)

        self.trail.append(pos)
        if len(self.trail) > 15:
            self.trail.pop(0)

    def _slice(self, fruit):
        fruit.sliced = True
        if fruit.kind == "bomb":
            self.game_over = True
        else:
            self.score += 1

    def handle_input(self):
        pass

    def update(self):
        if self.game_over:
            return

        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.spawn_fruit()

        still_alive = []
        for fruit in self.fruits:
            fruit.update()
            if fruit.sliced:
                continue
            if fruit.off_screen(self.height):
                if fruit.kind == "fruit":
                    self.lives -= 1
                continue
            still_alive.append(fruit)
        self.fruits = still_alive

        if self.lives <= 0:
            self.game_over = True

    def render(self, screen):
        for fruit in self.fruits:
            color = getattr(fruit, "color", WHITE)
```

```
            pygame.draw.circle(screen, color, (int(fruit.x), int(fruit.y)), fruit

        if len(self.trail) >= 2:
            pygame.draw.lines(screen, WHITE, False, self.trail, 3)

        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))
        lives_text = self.font.render(f"Lives: {self.lives}", True, WHITE)
        screen.blit(lives_text, (self.width - 130, 10))

        if self.game_over:
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            go_surf = self.title_font.render("GAME OVER", True, RED)
            go_rect = go_surf.get_rect(center=(self.width // 2, 110))
            screen.blit(go_surf, go_rect)

            score_surf = self.font.render(f"Final Score: {self.score}", True, WHI
            score_rect = score_surf.get_rect(center=(self.width // 2, 175))
            screen.blit(score_surf, score_rect)

            prompt_surf = self.font.render("Play Again - Select Difficulty:", Tru
            prompt_rect = prompt_surf.get_rect(center=(self.width // 2, 235))
            screen.blit(prompt_surf, prompt_rect)

            self.button_rects = {}
            btn_width, btn_height = 260, 48
            btn_x = self.width // 2 - btn_width // 2

            options = [
                ("EASY", "1. Easy (5% Bombs)", 285),
                ("MEDIUM", "2. Medium (15% Bombs)", 345),
                ("HARD", "3. Hard (30% Bombs)", 405),
            ]

            mouse_pos = pygame.mouse.get_pos()

            for key, label, y_pos in options:
                rect = pygame.Rect(btn_x, y_pos, btn_width, btn_height)
                self.button_rects[key] = rect

                is_selected = (self.difficulty == key)
                is_hovered = rect.collidepoint(mouse_pos)
                bg_color = (60, 70, 90) if not (is_selected or is_hovered) else (
```

```
            border_color = DIFFICULTIES[key]["color"] if is_selected else WHI

            pygame.draw.rect(screen, bg_color, rect, border_radius=8)
            pygame.draw.rect(screen, border_color, rect, width=2 if not is_se

            txt_surf = self.button_font.render(label, True, WHITE)
            txt_rect = txt_surf.get_rect(center=rect.center)
            screen.blit(txt_surf, txt_rect)

        # Draw Exit Button
        exit_rect = pygame.Rect(btn_x, 475, btn_width, btn_height)
        self.button_rects["EXIT"] = exit_rect
        exit_hover = exit_rect.collidepoint(mouse_pos)
        exit_bg = (120, 40, 40) if exit_hover else (70, 30, 30)

        pygame.draw.rect(screen, exit_bg, exit_rect, border_radius=8)
        pygame.draw.rect(screen, RED, exit_rect, width=2, border_radius=8)

        exit_surf = self.button_font.render("Exit Game (Esc)", True, WHITE)
        exit_rect_txt = exit_surf.get_rect(center=exit_rect.center)
        screen.blit(exit_surf, exit_rect_txt)
```

Add basic sound effects for slicing a fruit, hitting a bomb, and the game-over moment

Here are the updated `game_engine.py` and `main.py` files with procedurally generated sound effects for slicing fruits, hitting a bomb, and the game-over moment using Pygame's `sndarray` module (no external audio assets required). PY + 1

Added Sound Effects

1. **Slice Sound**: A short high-pitched frequency sweep (blade "swisss" sound).
2. **Bomb Sound**: A low-frequency decaying noise burst (explosion sound).
3. **Game Over Sound**: A descending dual-tone sad sound.

1. Updated `game_engine.py`

Python

```
), (230, 140, 40), (230, 200, 40), (90, 180, 90)]


 0.05, "color": GREEN},
': 0.15, "color": ORANGE},
 0.30, "color": RED},



 sound effects using numpy audio arrays."""

4100
ie.mixer.get_init() is not None


l = self._create_slice_sound()
 = self._create_bomb_sound()
ound = self._create_game_over_sound()

lf, samples):
nteger PCM array
mples * 32767)
ray to 2D stereo array for Pygame mixer
tack((scaled, scaled))
ay.make_sound(stereo)

self):
econds
duration, int(self.sample_rate * duration), False)
om 800 Hz down to 200 Hz
00, 200, len(t))
```

```
: * 30)
: np.pi * freq * t) * envelope
:_to_sound(samples)

:elf):

duration, int(self.sample_rate * duration), False)
on effect
niform(-1.0, 1.0, len(t))
: * 8)
nvelope
:_to_sound(samples)

ound(self):

duration, int(self.sample_rate * duration), False)
one jingle
: 0.3, 300, 200)
: 0.3, 240, 160)
: * 3)
 sin(2 * np.pi * freq1 * t) + 0.5 * np.sin(2 * np.pi * freq2 * t)) * envelope
:_to_sound(samples)



l.play()



 play()

 :

:ound.play()



n, height):

:



 = 55
 0
```

```
IEDIUM"
)IFFICULTIES[self.difficulty]["bomb_chance"]
..0



'ont.SysFont("Arial", 28)
'game.font.SysFont("Arial", 50, bold=True)
)ygame.font.SysFont("Arial", 20, bold=True)

.se
'alse
{}

lanager()

 mode):
'IES:
 = mode
 = DIFFICULTIES[mode]["bomb_chance"]

:ulty=None):

ılty(difficulty)




0
.se


)0, self.width - 60)
ı(13, 16) * self.speed_scale
-2, 2)

ıdom.random() < self.bomb_chance else "fruit"

.f.height + 30, vx, vy, gravity, kind=kind)
3LACK if kind == "bomb" else random.choice(FRUIT_COLORS)
'ruit)

:vent):

 pygame.MOUSEBUTTONDOWN and event.button == 1:
 pos
```

```
:t in self.button_rects.items():
 collidepoint(pos):
:ey in DIFFICULTIES:
 self.restart(difficulty=key)
: key == "EXIT":
 self.should_quit = True
 == pygame.KEYDOWN:
' == pygame.K_1:
:tart("EASY")
:ey == pygame.K_2:
:tart("MEDIUM")
:ey == pygame.K_3:
:tart("HARD")
:ey in (pygame.K_ESCAPE, pygame.K_q):
)uld_quit = True


;ame.MOUSEMOTION:
:ion(event.pos)

  pos):

 trail[-1]
.f.fruits:
:.sliced and fruit.intersects_segment(last_pos, pos):
.ice(fruit)

.f.fruits:
:.sliced and fruit.contains_point(pos[0], pos[1]):
.ice(fruit)

)s)
 15:
))


)mb":
'_bomb()
ıme_over()

'_slice()


:elf):
:r:
```

```
: True
'_game_over()
```

```
: 1
' >= self.spawn_interval:
:r = 0
:()
```

```
'uits:
```

```
'een(self.height):
id == "fruit":
'es -= 1

:nd(fruit)
.alive
```

```
ime_over()

 :
'uits:
(fruit, "color", WHITE)
:le(screen, color, (int(fruit.x), int(fruit.y)), fruit.radius)

:= 2:
:s(screen, WHITE, False, self.trail, 3)

ont.render(f"Score: {self.score}", True, WHITE)
:xt, (10, 10))
ont.render(f"Lives: {self.lives}", True, WHITE)
:xt, (self.width - 130, 10))
```

```
:.Surface((self.width, self.height), pygame.SRCALPHA)
 0, 0, 200))
```

```
lay, (0, 0))

itle_font.render("GAME OVER", True, RED)
f.get_rect(center=(self.width // 2, 110))
urf, go_rect)

f.font.render(f"Final Score: {self.score}", True, WHITE)
re_surf.get_rect(center=(self.width // 2, 175))
e_surf, score_rect)

lf.font.render("Play Again - Select Difficulty:", True, YELLOW)
ompt_surf.get_rect(center=(self.width // 2, 235))
pt_surf, prompt_rect)

s = {}
eight = 260, 48
lth // 2 - btn_width // 2


 Easy (5% Bombs)", 285),
2. Medium (15% Bombs)", 345),
 Hard (30% Bombs)", 405),


me.mouse.get_pos()

y_pos in options:
e.Rect(btn_x, y_pos, btn_width, btn_height)
rects[key] = rect

= (self.difficulty == key)
 rect.collidepoint(mouse_pos)
60, 70, 90) if not (is_selected or is_hovered) else (90, 110, 140)
 = DIFFICULTIES[key]["color"] if is_selected else WHITE

rect(screen, bg_color, rect, border_radius=8)
rect(screen, border_color, rect, width=2 if not is_selected else 4, border_radius

elf.button_font.render(label, True, WHITE)
xt_surf.get_rect(center=rect.center)
txt_surf, txt_rect)

me.Rect(btn_x, 475, btn_width, btn_height)
s["EXIT"] = exit_rect
t_rect.collidepoint(mouse_pos)
40, 40) if exit_hover else (70, 30, 30)
```

```
:(screen, exit_bg, exit_rect, border_radius=8)
:(screen, RED, exit_rect, width=2, border_radius=8)

:.button_font.render("Exit Game (Esc)", True, WHITE)
 exit_surf.get_rect(center=exit_rect.center)
:_surf, exit_rect_txt)
```

2. Updated `main.py`

Python

```python
import pygame
from game.game_engine import GameEngine

# Initialize pygame & audio mixer
pygame.mixer.pre_init(44100, -16, 2, 512)
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 700, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Fruit Slice - Pygame Version")

# Colors
DARK_BLUE = (20, 25, 45)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(DARK_BLUE)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
```

```
            engine.handle_event(event)

        if engine.should_quit:
            running = False

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if                "    .    "
```